

РАЗРЕШЕНИЕ ИМЁН В ЯЗЫКАХ ПРОГРАММИРОВАНИЯ НА ОСНОВЕ СВЕДЕНИЯ СТЕКОВЫХ ГРАФОВ К КОНТЕКСТНО-СВОБОДНОЙ ДОСТИЖИМОСТИ

Ильин Е. С.¹ ✉ egor2105094ilin@gmail.com

Григорьев С. В.² s.v.grigoriev@spbu.ru, orcid.org/0000-0002-7966-0698

¹Санкт-Петербургский государственный электротехнический университет «ЛЭТИ» им. В. И. Ульянова (Ленина), Российская Федерация, 197022, Санкт-Петербург, ул. Профессора Попова, 5

²Санкт-Петербургский государственный университет, Российская Федерация, 199034, Санкт-Петербург, Университетская наб., 7–9

Аннотация

Работа посвящена абстрактному разрешению имён, то есть связыванию ссылок на символы с их определениями, с использованием графовых моделей. Рассматривается способ сведения модели стековых графов к задаче достижимости с контекстно-свободными ограничениями (КС-достижимости). Области применения являются инструменты анализа кода и навигации по кодовым базам, интегрированные среды разработки, компиляторы. В статье представлено теоретическое сведение модели стековых графов к КС-достижимости, рассмотрены его ограничения. Произведена проверка предложенного решения на практике, выполнено экспериментальное сравнение со стековыми графами. Также освещены направления дальнейших исследований и доработки сведения и разработанного программного средства.

Ключевые слова: *стековые графы, контекстно-свободная достижимость, абстрактное разрешение имён, алгоритмы на графах*

1. ВВЕДЕНИЕ

Разрешение имён – ключевая функция фронт-энда компилятора и современных сред разработки (IDE). Она применяется для перехода к определению, поддержки рефакторинга, семантической подсветки [?, ?]. На практике большинство реализаций остаётся привязано к конкретным языкам и носит характер ad hoc решений. Это затрудняет переносимость инструментов и построение формальных доказательств их корректности [?, ?]. В работах по графам областей видимости (Scope Graphs) предложена модель связывания имён, отделяющая структуру AST от правил построения графа областей видимости (Scope Graph) и семантики разрешения имён. Данная модель стала основой для декларативных спецификаций и реализации разрешения имён в коде [?, ?, ?]. Для больших кодовых баз

Ilin E. S., Grigorev S. V.

Name Resolution Based on Stack Graphs Conversion to Context-Free Reachability for Programming Languages.

© Computer Tools in Education, –. No –. P. 1–12.

предложены стековые графы (Stack Graphs) с инкрементальным построением графа по файлам и стековым алгоритмом поиска определений [?].

Задача достижимости с ограничениями в виде контекстно-свободных языков даёт фундамент для решения многих прикладных задач анализа кода программ, таких как межпроцедурный анализ указателей, анализ алиасов и других [?, ?, ?]. Задача КС-достижимости состоит в поиске всех пар вершин в графе с метками на рёбрах, между которыми существует такой путь, что слово составленное из меток его рёбер принадлежит заданному контекстно-свободному языку. Есть основания полагать, что часть шаблонов разрешения имён также может быть сведена к задаче достижимости с контекстно-свободными ограничениями. Целью данной работы является повышение эффективности (выражаемой в меньших затратах по времени и по памяти) механизмов разрешения имён за счёт теоретического и практического исследования сводимости стековых графов к КС-достижимости.

2. СТЕКОВЫЕ ГРАФЫ И КС-ДОСТИЖИМОСТЬ

В качестве примера рассматривается следующий код на языке программирования Java:

```
# other.java                                # main.java
class Y {                                    class HelloWorld {
    public int x;                            public static void main() {
    public Y() { this.x = 0; }                Y y;
}                                              y.x;
                                              }
                                              }
```

Листинг 1. Пример исходного кода

Пусть требуется разрешить ссылку на символ x в выражении $y.x$ в *main.java*. x является полем объекта y , следовательно, необходимо сначала разрешить ссылку y . Определение находится строкой выше и указывает, что y имеет тип Y . Необходимо разрешить ссылку на символ Y . Определение Y находится в файле *other.java* и содержит определение x , которое и является решением задачи.

Для формального обоснования предлагаемого метода вводятся определения стековых графов и задачи КС-достижимости.

2.1. Стековые графы

Для решения задачи разрешения имён может применяться модель стековых графов.

Стековый граф G является представлением программы P . Программа P состоит из множества файлов исходного кода F_i . Каждый файл исходного кода представим в виде множества синтаксических вершин (вершин синтаксического древа). В данном множестве синтаксических вершин можно выделить два подмножества (возможно пересекающихся): подмножество вершин, отражающих определения, и подмножество вершин, отражающих ссылки в исходном коде.

Пусть символ x – идентификатор из исходного кода, отражающий имя некой сущности в программе (или часть такого имени), либо символ оператора (например, оператор доступа к члену объекта $.$ или оператор вызова $()$).

Каждая вершина стекового графа G принадлежит к одному из четырёх типов:

1. Вершина области видимости (Scope node);
2. Корневая вершина (Root node);
3. Вершина добавления символа x на стек (Push node, далее вершина добавления);
4. Вершина удаления символа x со стека (Pop node, далее вершина удаления).

Дополнительно, каждая вершина N_i кроме корневой относится ровно к одному файлу исходного кода F_i . Часть вершин стекового графа G относится к конкретным синтаксическим вершинам данного файла F_i . Вершина добавления N_i называется вершиной ссылки, если она относится к синтаксической вершине, отражающей ссылку. Вершина удаления N_i называется вершиной определения, если она относится к синтаксической вершине, отражающей определение.

Пусть E_G – множество рёбер стекового графа G_i . Каждое ребро стекового графа направлено и соединяет входную вершину N_i с выходной вершиной N'_i . Рёбра могут соединять только вершины, принадлежащие одному файлу исходного кода F_i [?].

Путь p в стековом графе G состоит из начальной вершины N_i , конечной вершины N'_i и стека символов $\hat{x}$. Путь называется завершённым, если начальная вершина пути является вершиной ссылки, конечная вершина пути является вершиной определения и стек символов пути пуст. Каждое разрешение имени в программе P выразимо в виде завершённого пути в соответствующем данной программе стековом графе G [?].

Пустым путём называется путь, не содержащий рёбер. Такой путь создаётся путём поднятия вершины стекового графа. Поднятие вершины добавления кладёт соответствующий ей символ x на стек символов. Вершина удаления не может быть поднята. Другие вершины дают пустой путь с пустым символьным стеком.

Любой путь p может быть расширен путём добавления к нему ребра E_i , исходящего из конечной вершины данного пути p . Выходная вершина N'_i этого ребра E_i становится новой конечной вершиной пути p . Символьный стек пути p меняется в зависимости от новой конечной вершины N'_i . Если вершина N'_i является корневой вершиной или вершиной области видимости, то стек символов $\hat{x}$ не меняется. Если вершина N'_i является вершиной добавления, то соответствующий ей символ x кладётся на стек символов $\hat{x}$. Если вершина N'_i является вершиной удаления, то расширение пути возможно только в том случае, когда стек символов $\hat{x}$ не пуст и на его вершине лежит символ x , которому вершина удаления N'_i соответствует. В таком случае этот символ x снимается с вершины стека символов $\hat{x}$.

Если конечная вершина пути является корневой вершиной, то данный путь можно расширить до корневой вершины в подграфе любого другого файла (можно сказать, что корневая вершина принадлежит одновременно всем файлам). Это единственный способ проложить путь из одного файла в другой [?].

На основе определения завершённого пути, способов создания пустого пути и способов расширения пути можно построить алгоритм разрешения имён в стековом графе.

Пусть дана программа P и ссылка в данной программе. Требуется найти все определения в программе P , соответствующие данной ссылке.

Строится стековый граф G , соответствующий программе P . Пустой путь p создаётся поднятием вершины, соответствующей данной ссылке. Затем от этой вершины выполняется обход в ширину. Очередь ожидающих путей изначально содержит только p . На

каждом шаге следующий путь извлекается из очереди, после чего к нему поочерёдно добавляются рёбра, исходящие из его конечной вершины. Если добавление допустимо по описанным выше правилам, новый путь помещается в очередь. Если найденный путь завершён, его конечная вершина соответствует искомому определению [?].

На рис. 1 показан стековый граф для кода из листинга 1. Вершины помещения (*push*) и извлечения (*pop*) обозначают соответственно добавление указанного символа на вершину стека и снятие верхнего символа при совпадении. Вершины областей видимости (*scope*) представляют области видимости. Корневая вершина (*root*) обеспечивает переход между файловыми подграфами. Ориентированные рёбра задают допустимые продолжения пути. Символы «.» и «:» введены правилами построения графа для Java: «.» моделирует обращение к члену объекта, а «:» обозначает запрос типа выражения. Зелёным выделен завершённый путь, связывающий ссылку *x* в выражении *u.x* с определением поля *x* класса *Y*.

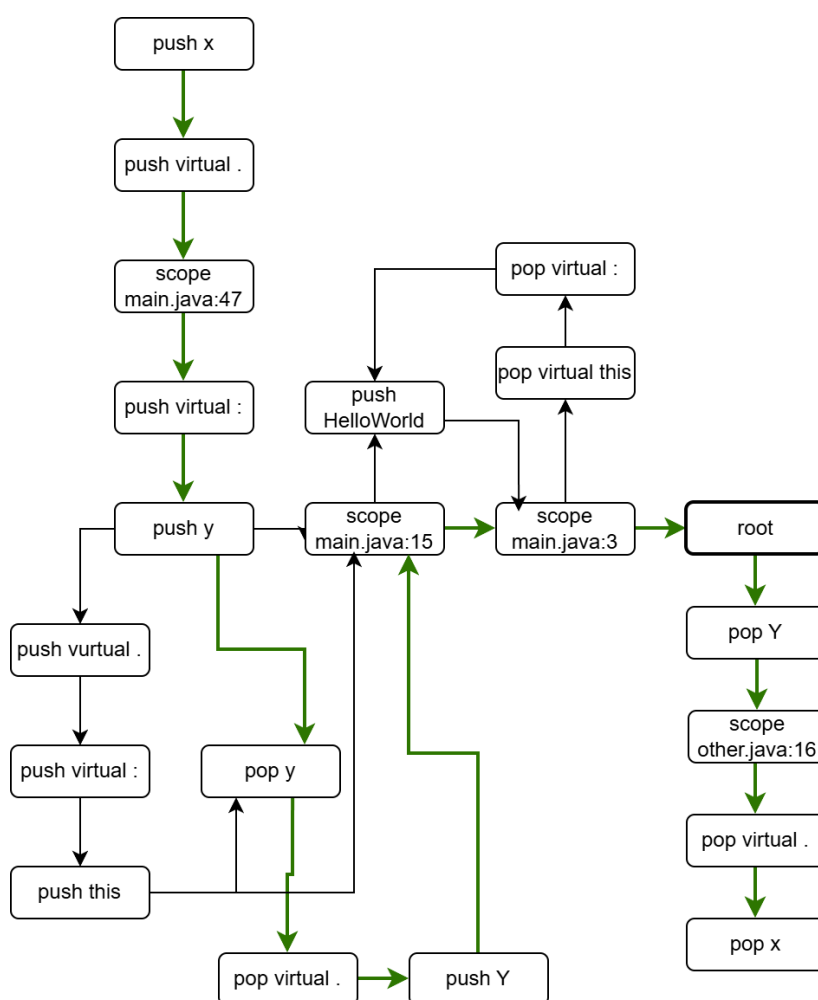


Рис. 1. Пример стекового графа

Граф строится из локальных фрагментов, создаваемых по правилам для синтаксических конструкций анализируемого языка (в нашем случае, для Java). Рёбра связывают эти фрагменты через вершины областей видимости в соответствии с правилами разрешения и вложенности областей. Ссылка *x* в выражении *u.x* образует начальную вершину

ну $push_x$. Доступ к члену объекта добавляет внутреннюю вершину $push$ и направляет поиск к типу объекта y . Для получения типа идентификатора правила Java создают переход через $push$ к ссылке $push_y$. Объявление Y представлено фрагментом $pop_y \rightarrow pop \rightarrow push_y$. Вершина pop_y сопоставляет ссылку с объявлением переменной, pop завершает запрос её типа, а $push_y$ начинает разрешение имени объявленного типа. Через области видимости файла *main.java* и корневую вершину путь переходит в подграф файла *other.java*, где pop_y сопоставляет имя типа с объявлением класса. Затем путь входит в область видимости членов класса: pop снимает маркер обращения к члену, а pop_x сопоставляет исходную ссылку с определением поля.

Зелёный путь показывает последовательность разрешения. Если вершину стека располагать слева, состояния символьного стека после операций *push* и *pop* имеют следующий вид:

$$\begin{aligned} [x] \rightarrow [., x] \rightarrow [:, ., x] \rightarrow [y, :, ., x] \rightarrow [:, ., x] \\ \rightarrow [., x] \rightarrow [Y, ., x] \rightarrow [., x] \rightarrow [x] \rightarrow []. \end{aligned}$$

Вершины областей видимости и корневая вершина стек не изменяют. Пустой стек после прохождения конечной вершины pop_x показывает, что путь завершён и связывает ссылку x с определением поля x класса Y .

Числа в вершинах областей видимости являются идентификаторами областей видимости данного файла. Граф на рис. 1 визуально сокращён: на схеме показана часть вершин и рёбер полного графа, достаточная для разбора выделенного пути. Полный граф существенно больше из-за дополнительных вершин областей видимости и ответвлений.

2.2. Задача КС-достижимости

Задача КС-достижимости определяется тремя объектами: граф G , алфавит Σ и контекстно-свободный язык L , заданный над алфавитом Σ .

Граф G состоит из множества вершин V и множества рёбер E , где $E \subseteq V \times (\Sigma \cup \{\varepsilon\}) \times V$. Рёбра ориентированы и размечены символами алфавита Σ или меткой ε . Символы алфавита далее называются терминалами.

Путь p в графе представлен последовательностью рёбер $E_1 \dots E_n$. Пути p соответствует слово w_p , образованное последовательной записью меток этих рёбер. Метка ε не добавляет символа в слово.

Пара вершин (u, v) графа G называется КС-достижимой, если существует путь p из u в v , слово которого принадлежит языку L .

Решение задачи КС-достижимости сводится к нахождению всех КС-достижимых пар вершин (u, v) графа G [?].

3. СВЕДЕНИЕ СТЕКОВЫХ ГРАФОВ К КС-ДОСТИЖИМОСТИ

В разделе дано формальное описание разработанного метода сведения, указана область его применимости и описано программное средство, созданное для демонстрации метода.

3.1. Теоретическое описание сведения

Рассматривается конечный путь $p_S = (N_1, \dots, N_n)$ в стековом графе G_S . Операция, соответствующая начальной вершине, входит в слово пути. Если начальная вершина является вершиной-ссылкой, соответствующий символ помещается на стек.

Далее рассматривается модель, в которой допустимость пути определяется символьным стеком. Вершины областей видимости и корневая вершина этот стек не изменяют. Корневая вершина обеспечивает переход между файловыми подграфами.

Множество символов стекового графа обозначается $X = \{x_i \mid i \in I\}$. Вершине помещения символа x_i сопоставляется терминал $push_i$, а вершине извлечения того же символа сопоставляется терминал pop_i . Операция $push_i$ добавляет x_i к текущему стеку. Операция pop_i допустима только при наличии x_i на вершине стека и удаляет этот символ. Слово пути образуется последовательностью этих операций. Нейтральные вершины не добавляют символов в слово пути.

Слова фрагментов пути, оставляющих символьный стек неизменным, описывает нетерминал S :

$$S \rightarrow \varepsilon \mid SS \mid push_i S pop_i, \quad i \in I.$$

Для запросов разрешения имён, рассматриваемых далее, используется стартовый нетерминал Q :

$$Q \rightarrow push_i S pop_i, \quad i \in I.$$

Эти правила задают контекстно-свободный язык L_S . Правило $S \rightarrow SS$ допускает произвольное число последовательных согласованных фрагментов, а правило с одинаковым индексом i обеспечивает правильную вложенность операций. К терминалам относятся только $push_i$ и pop_i . Обозначения S и Q являются нетерминалами, а ε обозначает пустое слово.

Пусть R и D обозначают множества вершин-ссылок и вершин-определений. При сведении каждой вершине $r \in R$ сопоставляется вход r_{in} . Каждой вершине $d \in D$ сопоставляется выход d_{out} . В построенной задаче КС-достижимости пара (r, d) считается результатом запроса, если существует путь из r_{in} в d_{out} , слово которого выводится из стартового нетерминала Q .

В математической записи ε обозначает пустое слово. Ребро с меткой ε является нейтральным. Прохождение такого ребра не добавляет терминала в слово пути и не изменяет символьный стек.

При преобразовании обозначения $push_i$ и pop_i переносятся с вершин на рёбра. Преобразование выполняется следующим образом.

1. Вершинам помещения и извлечения сопоставляются метки $push_i$ и pop_i . Вершины областей видимости и корневая вершина остаются нейтральными.
2. На промежуточном этапе метки могут находиться как на вершинах, так и на рёбрах графа. Нейтральные элементы не добавляют символов в слово пути.
3. Все исходные рёбра получают нейтральную метку ε . Прохождение такого ребра не изменяет слово пути.
4. Нейтральные метки удаляются с вершин областей видимости и с корневой вершины. Удаление этих меток не изменяет слова путей.

5. Каждая вершина с меткой $push_i$ заменяется парой вершин без меток N_{in} и N_{out} . Входящие рёбра перенаправляются в N_{in} . Исходящие рёбра перенаправляются из N_{out} . Между N_{in} и N_{out} добавляется ребро с меткой $push_i$. Каждому исходному пути через заменяемую вершину соответствует единственный путь через созданную пару с той же операцией в той же позиции слова. Если исходная вершина была вершиной ссылки, N_{in} становится соответствующим входом r_{in} .
6. Аналогично заменяются вершины с меткой pop_i . Если исходный путь заканчивался в вершине-определении, преобразованный путь заканчивается в соответствующей вершине N_{out} , которая служит выходом d_{out} .

После преобразования обозначения $push_i$ и pop_i существуют только как метки рёбер. Нейтральные рёбра имеют метку ϵ . Алфавит терминалов состоит из $push_i$ и pop_i . Время построения графа составляет $O(|V| + |E|)$, поскольку каждая вершина и каждое ребро обрабатываются постоянное число раз.

Псевдокод описанного алгоритма:

```
fn stack_graph_to_cfl(G: Graph) -> Graph:
  for node in G.nodes:
    # Push(x) -> push_x, Pop(x) -> pop_x,
    # Root and Scope -> epsilon
    node.label = make_node_label(node)
  for edge in G.edges:
    edge.label = epsilon
  for node in G.nodes where node.label == epsilon:
    delete node.label
  for node in G.nodes where node.label is push_*:
    let (N_in, N_out) = G.create_nodes_pair()
    redirect_incoming_edges(old_dest: node, new_dest: N_in)
    redirect_outgoing_edges(old_src: node, new_src: N_out)
    G.create_edge(N_in, N_out, label: node.label)
    if G.start_node == node:
      G.start_node = N_in
    G.delete_node(node)
  # То же самое для pop-вершин
  return G
```

Пример замены вершины помещения приведён на рис. 3 и 4.

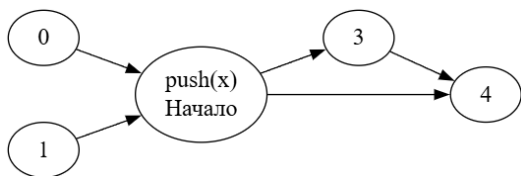


Рис. 3. Вершина помещения до преобразования

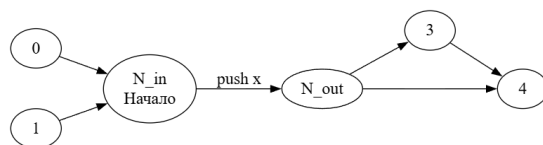


Рис. 4. Вершина помещения после преобразования

При разрешении всех ссылок рассматриваются входы r_{in} , соответствующие вершинам-ссылкам. В результат включаются все пары (r, d) , для которых существует путь из r_{in} в d_{out} , слово которого выводится из стартового нетерминала Q .

На рис. 5 показан результат сведения графа с рис. 1. Каждая вершина помещения или извлечения заменяется парой вершин N_{in} и N_{out} без меток, соединённых ребром с меткой соответствующей операции. Входящие рёбра перенаправляются в N_{in} . Исходящие рёбра перенаправляются из N_{out} . Вершины областей видимости и корневая вершина сохраняются, а исходные рёбра стекового графа получают нейтральную метку ε . Преобразование сохраняет разветвления, порядок прохождения и последовательность операций пути.

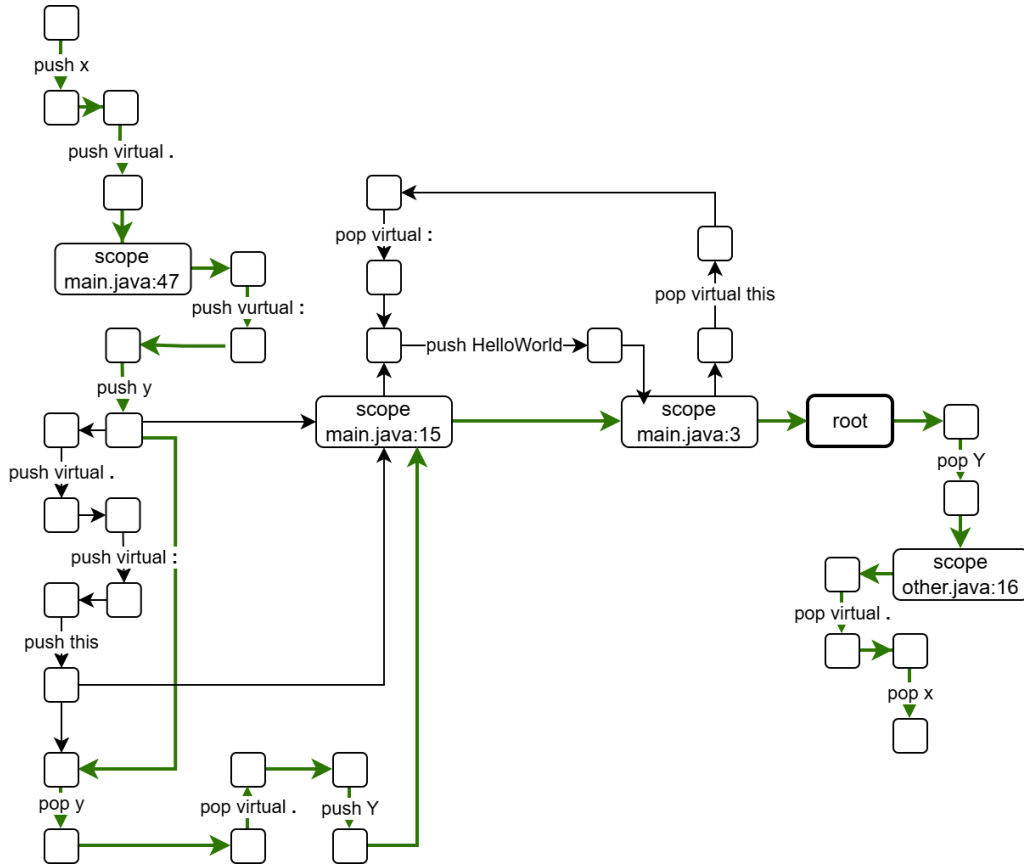


Рис. 5. Результат сведения стекового графа из рис. 1

Зелёный путь на рис. 5 соответствует зелёному пути на рис. 1. Если опустить нейтральные рёбра, его слово имеет вид

$$\begin{aligned} & push_x \rightarrow push_{\cdot} \rightarrow push_{\cdot} \rightarrow push_y \rightarrow pop_y \\ & \rightarrow pop_{\cdot} \rightarrow push_Y \rightarrow pop_Y \rightarrow pop_{\cdot} \rightarrow pop_x. \end{aligned}$$

Операции вложены и попарно согласованы: каждая операция pop снимает верхний совпадающий символ, поэтому слово выводится из Q . Вершина N_{in} пары, заменившей $push_x$, соответствует исходной ссылке, а вершина N_{out} пары, заменившей pop_x , соответствует определению. Тем самым сведение сохраняет связь ссылки x в выражении $u.x$ с определением поля x класса Y .

3.2. Программное средство StackGraphExporter

Для практической реализации описанного выше сведения стековых графов к задаче КС-достижимости разработана утилита командной строки StackGraphExporter [?] на языке Rust. Программное средство строит стековые графы по файлам исходного кода, выполняет запросы разрешения на стековом графе, сводит стековые графы к задаче КС-достижимости, при необходимости упрощает граф в процессе сведения и создаёт артефакты для последующего запуска внешних решателей.

В экспериментах библиотека stack-graphs 0.14.1 используется как эталонная реализация. Перед выполнением запросов библиотека для каждого файла строит минимальный набор частичных путей и помещает его в базу. Частичный путь представляет собой фрагмент пути разрешения, который можно состыковать с другими фрагментами. При разрешении ссылки библиотека объединяет совместимые частичные пути в полные пути от ссылки к определениям.

4. РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ

Для проверки корректности реализации сведения и измерения эффективности разработанного программного средства проведена серия экспериментов. В разделе приведены постановка экспериментов и результаты.

4.1. Постановка экспериментов

В эксперименте отобраны четыре проекта на Java с платформы GitHub: libgdx [?], Shattered Pixel Dungeon [?], JsonPath [?] и JiaoZi Video Player [?]. В качестве внешнего решателя использовался CFG_bench. В рамках каждого эксперимента выполнено 10 запусков. Средние значения времени запроса и потребляемой памяти рассчитаны средствами решателя. Для stack-graphs перед измерениями выполнялись два разогревочных запуска, а среднее значение рассчитывалось по пяти последующим запускам. Пиковое потребление ОЗУ в таблице приведено отдельно для stack-graphs и CFG_bench. Показатель для stack-graphs измерялся с помощью библиотеки peak_alloc. В каждом случае выполнялось разрешение всех ссылок в графе. Техническое окружение: ноутбук под управлением Windows 11 Pro, процессор AMD Ryzen 9 8940HX, 32 ГБ ОЗУ DDR5 SDRAM.

4.2. Проверка корректности

Для проверки корректности реализации сведения стекового графа к задаче КС-достижимости использовался решатель CFG_bench. В результате решения задачи КС-достижимости определялось количество найденных пар вершин. Полученное значение сравнивалось с количеством пар, найденных при запросах в стековом графе.

4.3. Результаты для запроса всех ссылок в графе

Результаты экспериментов приведены в табл. 1.

Табл. 1. Результаты выполнения запросов для всех пар

Проект	libgdx	Shattered Pixel Dungeon	JsonPath	JiaoZi Video Player
Время, стековый граф	278 с	–	2,6 с	1,13 с
Память, стековый граф	27 ГБ	–	3,7 ГБ	2,1 ГБ
Время построения КС-графа	8 с	3,8 с	39 мс	18 мс
Время построения КС-графа с упрощением	38 с	31 с	0,34 с	28 мс
Время, CFG_bench	–	–	6,9 с	1,67 с
Время, CFG_bench с упрощением	7 с	1,3 с	0,28 с	28 мс
Память, CFG_bench	–	–	1,3 ГБ	239 МБ
Память, CFG_bench с упрощением	1,23 ГБ	227 МБ	38 МБ	14 МБ

Примечания к табл. 1.

1. Пропуски в таблице обозначают неудавшиеся запуски измерений. Как правило, решателю не хватало доступной оперативной памяти.
2. Для реализации stack-graphs время указано как сумма времени построения базы частичных путей и времени выполнения запроса после её построения.
3. Для проекта libgdx запрос в стековом графе был разбит на части по 32 768 начальных вершин. Без разбиения большинство запусков завершалось ошибкой выделения памяти. В нескольких успешных запусках время выполнения без разбиения превышало время выполнения с разбиением на 30–40%. Для Shattered Pixel Dungeon запрос всех пар (ссылка, определение) не удалось завершить даже при обработке по одной начальной вершине.

Совместное использование упрощения и CFG_bench при разрешении всех ссылок сократило время выполнения запросов. Для Shattered Pixel Dungeon все ссылки удалось разрешить только при использовании CFG_bench с упрощением. Для остальных трёх проектов суммарное время выполнения уменьшилось на 76–95% по сравнению со stack-graphs. Пиковое потребление памяти при использовании CFG_bench было на 95–99% ниже, чем при использовании stack-graphs.

5. ВЫВОДЫ И НАПРАВЛЕНИЯ ДАЛЬНЕЙШИХ ИССЛЕДОВАНИЙ

В рамках работы обоснована перспективность применения КС-достижимости к задаче абстрактного разрешения имён. КС-достижимость применяется в смежных областях, в том числе в межпроцедурном анализе указателей. Для КС-достижимости получены теоретические результаты и разработаны оптимизированные реализации. Выбор стековых графов в качестве основы сведения обусловлен сходством двух моделей: в обоих случаях выполняется поиск путей в графе с ограничениями.

Формально выведен способ сведения стековых графов, в которых ограничения пути задаются символьным стеком, к задаче КС-достижимости. Сложность сведения составляет $O(|V| + |E|)$, где $|V|$ обозначает число вершин, а $|E|$ обозначает число рёбер стекового графа. Как показано в разд. 3.2, преобразование сохраняет концы и слово каждого конечного пути.

Разработан и реализован программный инструмент StackGraphExporter, позволяющий автоматически строить стековые графы по исходному коду на Java, выполнять запросы разрешения, преобразовывать стековый граф в задачу КС-достижимости, применять эвристики упрощения в процессе сведения и создавать артефакты для запуска внешнего решателя CFG_bench.

В рамках разработанного программного инструмента экспериментально проверена корректность сведения и исследована эффективность предложенных эвристик упрощения графа. Максимальное сокращение составило 79% для вершин КС-графа и 32% для рёбер. Для трёх проектов совместное использование упрощения и CFG_bench при разрешении всех ссылок сократило суммарное время выполнения на 76–95% и пиковое потребление памяти на 95–99% по сравнению со stack-graphs.

К направлениям дальнейших исследований относятся разработка инкрементального алгоритма для задачи КС-достижимости, чтобы минимизировать вычисления при изменении части файлов исходного кода. Для этого, в частности, необходимо исследовать возможности переиспользования результатов запросов и предвычисления путей для подграфов на уровне решателя КС-достижимости. Также представляют интерес разработка модели использования КС-достижимости без промежуточного стекового графа и оптимизация решателей КС-достижимости для разрешения имён.

Ильин Егор Сергеевич, выпускник кафедры математического обеспечения и применения ЭВМ, СПбГЭТУ «ЛЭТИ», ✉ egor2105094ilin@gmail.com

Григорьев Семён Вячеславович, канд. физ.-мат. наук, доцент кафедры системного программирования СПбГУ, s.v.grigoriev@spbu.ru

№ –: 1–12

<http://cte.eltech.ru>

Name Resolution Based on Stack Graphs Conversion to Context-Free Reachability for Programming Languages

Ilin E. S.¹ ✉ egor2105094ilin@gmail.com

Grigoriev S. V.² s.v.grigoriev@spbu.ru, orcid.org/0000-0002-7966-0698

¹Saint Petersburg State Electrotechnical University “LETI”, Professora Popova ul. 5, St. Petersburg, 197022, Russian Federation

²Saint Petersburg State University, Universitetskaya nab. 7–9, St. Petersburg, 199034, Russian Federation

Abstract

The problem of abstract name resolution (binding symbol references to corresponding symbol definitions) in graph models is covered in the article. The method for stack graphs model conversion to the problem of context-free reachability (CFL-reachability) is considered. The areas of applicability encompass code analysis and navigation tools, integrated development environments, compilers. The article presents conversion of stack graphs model to CFL-reachability and considers conversion's limitations. A practical check of the proposed solution and an experimental comparison with stack graphs are conducted. Additionally, further areas of research and improvement of the presented method and the designed program are highlighted.

Keywords: *stack graphs, context-free reachability, abstract name resolution, graph algorithms*

Ilin Egor, graduate of ETU “LETI”, ✉ egor2105094ilin@gmail.com

Semyon Grigorev, Associate Professor at St. Petersburg State University, s.v.grigoriev@spbu.ru